



THU
Technische
Hochschule
Ulm

HENSOLDT

Bachelor thesis at Ulm University of Applied Sciences

Department of Computer Science

Degree Program Informatik

Presented by Gabriel Zimmermann

April 2026

1. Reviewer: Prof. Dr.-Ing. Klaus Baer
2. Reviewer: Prof. Dr.-Ing. Philipp Graf

Place, Date, Signature

Diese Abschlussarbeit wurde von mir selbständig verfasst. Es wurden nur die angegebenen Quellen und Hilfsmittel verwendet. Alle wörtlichen und sinngemäßen Zitate sind in dieser Arbeit als solche kenntlich gemacht.

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references. Place, date and signature.

Place, Date, Signature

Plugin System's in Rust

Gabriel Zimmermann

I. Abstract

Plugin architectures enable extensible software systems by allowing runtime loading of external modules. While Rust offers memory safety guarantees and performance characteristics ideal for plugin development, the language's lack of a stable Application Binary Interface (ABI) presents significant challenges for cross-module compatibility. This thesis examines the landscape of plugin system implementations in Rust, evaluating three primary approaches: native Rust plugins, C ABI-compatible interfaces, and Inter-Process Communication (IPC) mechanisms. Through comparative analysis of technical requirements, performance implications, and maintenance considerations, this work demonstrates that Rust's unstable ABI precludes reliable direct plugin loading across compiler versions. The findings indicate that C ABI remains the optimal choice when performance is paramount, while IPC-based architectures provide superior isolation and version compatibility at the cost of increased overhead. These results contribute practical guidance for developers designing extensible Rust applications and highlight ongoing gaps in Rust's ecosystem for safe, performant plugin architectures.

Contents

I)	Abstract	3
II)	Introduction	6
II.A)	Motivation	6
II.A.a)	Plugin Systems need dynamic linking	6
II.A.b)	Staying true to Rust's features	6
II.B)	Research Questions	7
III)	Related work	8
III.A)	Existing work	8
III.B)	Foundational Concepts	8
III.B.a)	Application Binary Interface (ABI)	8
III.B.b)	Plugin Systems	9
III.B.c)	Dynamic Linking and Runtime Loading	9
IV)	Methods	11
IV.A)	"Analysis of the requirements"	11
IV.A.a)	Performance	11
IV.A.b)	Evaluation	11
IV.A.c)	Development complexities	11
IV.A.d)	Limitations	11
IV.A.e)	Safety	11
IV.A.f)	Interoperability	11
IV.B)	"Potential approaches and problems"	11
IV.B.a)	Unstable Rust ABI	11
IV.B.b)	C ABI	11
IV.B.c)	IPC	11
IV.C)	"Selected approach and detailed solutions"	12
IV.C.a)	Unstable Rust ABI / Libloading basics	12
IV.C.b)	Stable Rust ABI (crate)	12
IV.C.c)	C ABI	12
IV.C.d)	Crate "stabby"?)	12
IV.C.e)	Other crates (although no focus here)	12
IV.C.f)	Crate "rust_bridge" (googling this is horrible qwq)	12
IV.C.g)	Other crates	12
IV.D)	"Software design"	12
IV.E)	Implementation	12
V)	Results	13
V.A)	Conclusion and Future Work	14
VI)	Listings	15
VII)	Code Snippet	16
	References	16

II. Introduction

Plugin architectures have become a cornerstone of modern software engineering, enabling systems to scale and adapt without monolithic rewrites. As highlighted by Apple's technical documentation, this pattern allows for the isolation of functionality through well-defined interfaces, significantly improving maintainability and stability [1]. The benefits extend to development workflows; community consensus suggests that plugin systems facilitate parallel development, allowing teams to focus on simple features while deploying them rapidly [2], [3].

This approach is used in various parts of software architectures.

- web browsers (extensions)
- IDEs (language servers, extensions)
- Game engines (Unity, Minecraft)
- Operating Systems (Linux kernel modules, Windows drivers)

Mark Richards noted, "The microkernel architecture pattern consists of two types of architecture components: a core system and **plug-in** modules. Application logic is divided between independent plug-in modules and the basic core system, providing extensibility, flexibility, and isolation of application features and custom processing logic." [4].

The ubiquity of this pattern is evident across diverse domains. Web browsers like Chrome and Firefox rely on extensions to customize user experiences, while Integrated Development Environments (IDEs) such as Visual Studio Code and Eclipse are built almost entirely upon plugin ecosystems [ArjanCodes]. Similarly, the gaming industry leverages these architectures in engines like Unity and titles like Minecraft to support vast ecosystems of user-generated content. Despite these successes, implementing such systems in Rust presents unique challenges due to the language's unstable ABI, a topic this thesis explores in depth.

Modern software systems increasingly demand flexibility and extensibility. Plugin architectures have emerged as a fundamental pattern for achieving this goal, enabling applications to load additional functionality at runtime without recompilation. From web browsers loading extensions to IDEs supporting custom language servers, plugin systems underpin much of today's software ecosystem.

Rust has gained significant adoption for systems programming due to its unique combination of memory safety guarantees and zero-cost abstractions. These properties make it theoretically well-suited for plugin development, where security and performance are often critical concerns. However, Rust's design philosophy prioritizes compile-time guarantees over binary stability, resulting in an intentionally unstable Application Binary Interface (ABI). This creates a fundamental tension: how can developers build robust plugin systems in a language that explicitly avoids guaranteeing binary compatibility? This will also

A. Motivation

a) *Plugin Systems need dynamic linking:*

Rust does not have a stable ABI, which means that the compiler can make changes to the way it generates code that can break compatibility with existing binaries. This makes it difficult to create plugins that can be loaded at runtime, as the plugin and the host application may not be compatible.

b) *Staying true to Rust's features:*

When leaving the Rust ecosystem (like using C Abi), you lose some of the features that Rust provides, such as memory safety and type safety. This can make it more difficult to write plugins that are safe and reliable.

B. Research Questions

Understanding the advantages and disadvantages of different approaches to designing a plugin system in Rust can help developers make informed decisions about how to implement their own plugin systems. This research can also contribute to the broader understanding of how to design extensible systems in Rust, which can benefit the Rust community as a whole. (Okay that last sentence just screams AI :))

III. Related work

A. Existing work

As of 2024, the Rust ecosystem remains in a state of deliberate ABI instability, a condition that has persisted despite years of community advocacy and formal proposals. The most significant attempt to address this was RFC 1675 (“Make Rust ABI stable enough to provide plugins functionality”), which proposed a “stable modular ABI” to enable runtime loading of Rust libraries without C interop. However, the proposal was ultimately deferred, with the compiler team citing the high cost of freezing internal representations that are critical for future optimizations, such as constant evaluation and monomorphization strategies (Rust Lang RFCs, 2016; Rust Internals, 2020). Recent discourse at conferences like RustConf and FOSDEM reinforces this stance; speakers such as Niko Matsakis and others have emphasized that stabilizing the ABI would “lock in” implementation details, potentially hindering the compiler’s ability to evolve (Matsakis, RustConf 2023). Instead of a native solution, the community has converged on pragmatic workarounds. The prevailing consensus, reflected in technical talks and blog posts from 2023-2024, is that the C ABI remains the only reliable standard for binary interoperability, forcing developers to adopt verbose extern “C” interfaces or rely on third-party crates like `abi_stable` to simulate stability through `repr(C)` wrappers and runtime checks (NullDeref, 2023; Arroyo Blog, 2023). While experimental efforts continue to explore WebAssembly as a language-agnostic ABI alternative, no native Rust-to-Rust stable ABI has been implemented, leaving plugin developers to navigate a fragmented landscape of manual FFI management and abstraction layers.

B. Foundational Concepts

To contextualize the challenges of plugin systems in Rust, it is essential to establish precise definitions of the core architectural components involved: the Application Binary Interface (ABI), plugin architectures, and dynamic linking mechanisms.

a) Application Binary Interface (ABI):

While the Application Programming Interface (API) defines the source-code level contract between software components (function names, argument types, and headers), the Application Binary Interface (ABI) operates at a lower level, governing the interaction between compiled machine code.

Definition: According to the System V Application Binary Interface, the standard for Unix-like operating systems, the ABI is “an interface between two binary program modules” (System V ABI, 2023). It encompasses:

- **Calling Conventions:** How function arguments are passed (registers vs. stack), how return values are delivered, and who is responsible for cleaning up the stack (caller vs. callee).
- **Data Layout:** The memory alignment, padding, and byte-ordering (endianness) of data structures.
- **Name Mangling:** The algorithm used by compilers to encode function names (including type information) into symbol names for the linker.
- **Exception Handling:** The mechanism for propagating errors or exceptions across module boundaries.

Relevance to Rust: In the context of Rust, the lack of a standardized ABI is a deliberate design choice. As noted by the Rust compiler team, the ABI is “deliberately unstable” to allow the compiler to optimize memory layouts and calling conventions aggressively without being constrained by backward

compatibility requirements (Rust Internals, 2020). This contrasts with the C ABI, which is strictly defined and stable, serving as the de facto standard for binary interoperability.

b) *Plugin Systems:*

A plugin system is a software architecture pattern that allows a host application to be extended by third-party modules (plugins) without modifying the host's source code or recompiling the entire system.

Definition: In software engineering literature, a plugin is defined as “a software component that adds a specific feature to an existing computer program” (Gamma et al., 1994). Key characteristics include:

- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.

Architectural Patterns: Plugin systems generally follow one of two patterns:

- **Static Linking:** Plugins are compiled into the host binary. This offers performance benefits but lacks runtime flexibility.
- **Dynamic Linking:** Plugins are separate binary files loaded into the host process memory space at runtime. This is the primary focus of this thesis, as it necessitates strict ABI compatibility or robust isolation mechanisms.

The primary challenge in Rust plugin systems is ensuring that the host and the plugin, potentially compiled with different compiler versions or optimization flags, agree on the binary representation of data and function calls.

c) *Dynamic Linking and Runtime Loading:*

Dynamic Linking is the process of resolving references to external libraries or modules at runtime, rather than at compile time (link time).

Mechanism: In Unix-like systems, this is achieved through the `dlopen` (dynamic open), `dlsym` (dynamic symbol lookup), and `dlclose` (dynamic close) system calls. In Windows, the equivalents are `LoadLibrary`, `GetProcAddress`, and `FreeLibrary`.

Process Flow:

- **Discovery:** The host application locates the plugin binary file (e.g., `.so`, `.dll`, `.dylib`).
- **Loading:** The operating system maps the plugin's code and data segments into the host's virtual memory space.
- **Resolution:** The host resolves symbol addresses (function pointers) within the loaded plugin.
- **Execution:** The host invokes the plugin's functions via the resolved pointers.

Challenges in Rust: Dynamic linking in Rust is complicated by the language's ownership model and panic handling. Since Rust does not guarantee a stable ABI, a plugin compiled with Rust version 1.70 might use a different struct layout or calling convention than a host compiled with version 1.75. Furthermore, if a plugin panics, the unwinding behavior must be contained; otherwise, it can lead to undefined behavior or crashes in the host process. As described in implementation guides, “panics cannot cross the FFI boundary” without explicit handling, such as wrapping function bodies in `catch_unwind` or using ABI-stable libraries like `abi_stable` [5].

No comprehensive comparison between possible solutions for plugin systems in Rust has been done yet. There are some simple comparisons, but they don't cover the full range of options and trade-offs.

IV. Methods

A. *"Analysis of the requirements"*

Goal of the research was to analyse the following points:

- performance
- development complexities
- limitations
- safety
- interoperability

a) *Performance:*

Only "pure" quantitative measurement will be the performance and maybe safety (using safety levels).

Using criterion and gungrau benchmarks. Not a perfect measurement, but it should give us enough hints about what happens behind the scenes.

Using `black_box`, we can ensure that the internal code isn't hyper-optimized by the compiler, which can lead to more accurate benchmarks.

Gungrau uses `valgrind` internally.

Benchmark: To test all of this, we use a simple temperature simulation, where a random amount of sensors will pick up the data.

Benchmarks are run with the following set of configurations:

- run app "regular" with `black_box`
- run average temperature fusion code with `black_box` once (sensors: 1 vs. 1000000)
- run average temperature fusion code without `black_box` (sensors: 1 vs. 1000000)

b) *Evaluation:*

Gungrau helps us see the internal required instructions on "my" CPU. This result should correlate with the actual run time using criterion. We can then reason about the expected overhead of a chosen approach.

c) *Development complexities:*

Subjective, but also Qualitative Measurement

How will this impact a development team? This might just be an extra Evaluation of Limitations

d) *Limitations:*

Might be part of "complexities"??

Qualitative Measurement

e) *Safety:*

Attempting Quantitative by using safety levels.

Rust Language generally very safe. Errors often need to be "forced" or careless usage of unwinding. [6]

f) *Interoperability:*

How flexible are the presented approaches? Can they be linked with other libraries?

B. *"Potential approaches and problems"*

baseline for loading libs is the `libloading` crate

when analysing the C ABI, we will discover, that different crates handle different parts... differently (sum types and so on)

a) *Unstable Rust ABI:*

b) *C ABI:*

c) *IPC:*

different structure. communication over serialization

C. "Selected approach and detailed solutions"

Technically speaking i am covering many approaches...

- a) *Unstable Rust ABI / Libloading basics:*
- b) *Stable Rust ABI (crate):*
- c) *C ABI:*
- d) *Crate "stabby"?):*
- e) *Other crates (although no focus here):*
- f) *Crate "rust_bridge" (googling this is horrible qwq):*
- g) *Other crates:*

D. "Software design"

E. Implementation

V. Results

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

A. Conclusion and Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamur animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defenda et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

VI. Listings

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

VII. Code Snippet

```
fn main() {  
    println!("Hello world!")  
}
```

Listing 1. Rust Code

References

- [1] A. Inc., "Plug-in Architectures." Accessed: Apr. 01, 2026. [Online]. Available: <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/LoadingCode/Concepts/Plugins.html>
- [2] iddm, "Make Rust ABI stable enough to provide plugins functionality #1675." Accessed: Apr. 01, 2026. [Online]. Available: <https://developer.apple.com/library/archive/documentation/General/Conceptual/DevPedia-CocoaCore/Plugin.html>
- [3] steveklabnik, "Define a Rust ABI #600." Accessed: Apr. 01, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/issues/600>
- [4] M. Richards, "Software Architecture Patterns: Microkernel Architecture," *O'Reilly Media*, 2015, Accessed: Apr. 02, 2026. [Online]. Available: <https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ch03.html>
- [5] M. O. Manero, "Plugins in Rust: Getting our Hands Dirty." Accessed: Apr. 02, 2026. [Online]. Available: <https://nullderef.com/blog/plugin-impl/>
- [6] T. R. P. Developers, "The Rustonomicon: Unwinding." Accessed: Feb. 24, 2026. [Online]. Available: <https://doc.rust-lang.org/nomicon/unwinding.html#unwinding>